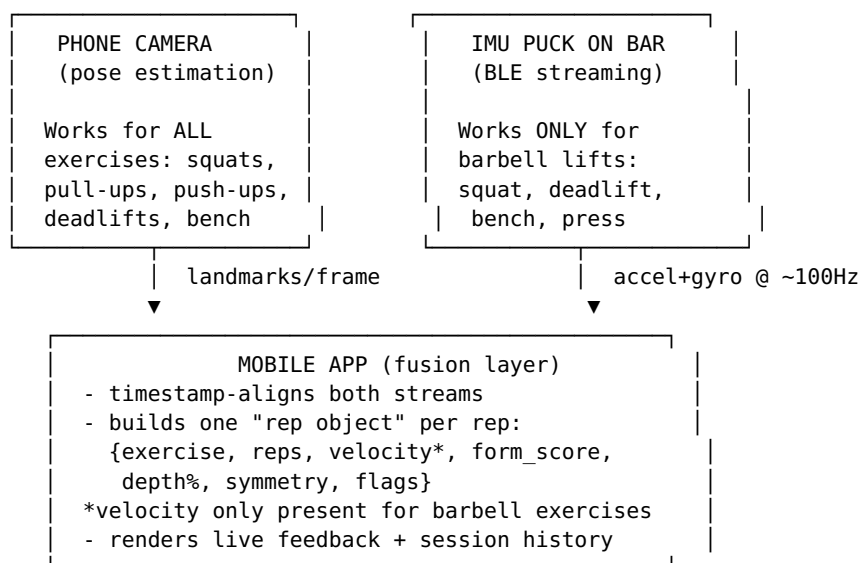


AI Gym Coach — Repo Reference & Integration Guide

Phone camera (pose/form, all exercises) + bar-mounted IMU (velocity, barbell lifts) fused into one app

1. System overview

Two independent sensing layers feed one app:



Key rule: **the IMU is barbell-only by physical necessity** (it clips to a bar). Pull-ups/push-ups/bodyweight work runs on the camera layer alone — there’s no “velocity” to fuse in for those, only form/rep data. The app should treat velocity as an *optional* field per exercise, not a required one.

2. Full repo reference

Layer A — Phone camera: pose estimation & rep counting (all exercises)

Repo	Link	Use i
Exercise-Counter	https://github.com/rushmash91/Exercise-Counter	Push-up, sc bicep-curl, counting + form corre MediaPipe math. Your path to a w multi-exerc counter. Push-up, pi squat via Y

AI-Gym-Trainer	https://github.com/shrishashegde/AI-Gym-Trainer	pose + Rep more robust counting w are occlude (e.g. pull-up overhead).
AI-Fitness-Trainer	https://github.com/samarthify/AI-Fitness-Trainer	Squats/pus ups/curls, r readable M angle-thres — best as a <i>reference</i> to write your
fitness-trainer-pose-estimation	https://github.com/yakupzengin/fitness-trainer-pose-estimation	Config-dri exercise en (Flask back web UI): ex defined in c hardcoded. the archit pattern yo app shoul
FormCheck	https://github.com/ctsc/FormCheck	Squat/dead rep countir 100 form s (depth, syn tempo, tors + spinal cu check. Best logic — por math to you exercises.
ai-workout-assistant	https://github.com/reevald/ai-workout-assistant	Push-up/sq TensorFlow MoveNet, r browser. U want a web with zero i judges.
FormFit	https://github.com/bigbaliboy/FormFit	Push-up/sq auto-class (MediaPipe ~95% repo accuracy) - app detect exercise is happening the user se
AI-FitnessTrainer (YOLOv8-pose)	https://github.com/KKopilka/AI-FitnessTrainer	Front squa up variants YOLOv8-pc this if Medi struggles v occlusion in footage.
pose-estimation-for-powerlifting	https://github.com/03y/pose-estimation-for-powerlifting	Precise squ + knee syn landmark r reference v need tighte formulas th general tra above.

Layer B — Bar-mounted IMU: velocity device (barbell lifts only)

Repo	Link
IMU-velocity-and-displacement-measurements	https://github.com/Wojtek120/IMU-velocity-and-displacement-measurements
OpenBarbell-V3	https://github.com/squatsandsciencelabs/OpenBarbell-V3
Smart-Bar	https://github.com/KevinAiken/Smart-Bar

Layer C — Optical velocity (no IMU needed — fallback or validation cross-check)

Repo	Link
BarbellTrackingCode (“Raise The Bar”)	https://github.com/dw2kim/BarbellTrackingCode
barbellcv	https://github.com/tlancon/barbellcv
VBT-Barbell-Tracker	https://github.com/kostecky/VBT-Barbell-Tracker

Layer D — Mobile app plumbing (BLE + firmware updates)

Repo	Link	Use it for
react-native-ble-plx	https://github.com/dotintent/react-native-ble-plx	BLE client library — connects the app to your IMU puck
react-native-ble-manager	https://github.com/innovent/react-native-ble-manager	Alternative BLE library if ble-plx gives you trouble on a specific device
react-native-nordic-dfu	https://github.com/Pilloxa/react-native-nordic-dfu	Push firmware updates to the nRF52840 over BLE without recalling hardware

3. Step-by-step integration

Step 0 — Decide your demo shape first

Pick one before writing code: - **Full system** (camera + IMU fused) — most impressive, most work, needs the physical puck built and paired
- **Camera-only** (Layer A, optionally C for barbell velocity) — zero hardware risk, faster to a working demo, still covers “not limited to barbell” fully

Everything below assumes you’re building toward the full system, but Steps 1 and 3 alone give you a complete camera-only demo if you’re short on time.

Step 1 — Stand up the camera/pose layer first (get this working before touching hardware)

1. Clone **yakupzengin/fitness-trainer-pose-estimation** as your base — its config-driven architecture (exercise → landmark set → angle thresholds defined in a config file, not code) is what lets you add pull-ups/push-ups/squats without rewriting core logic each time.
2. Pull the **rep-counting state machine** logic from **rushmash91/Exercise-Counter** (it already has a working pull-up counter) and the **form-scoring math** from **ctsc/FormCheck** (depth %, symmetry, torso angle, tempo) — merge both into yakupzengin’s config-driven engine so every exercise gets both a rep count *and* a form score, not just a count.
3. Test on webcam footage for each target exercise (squat, pull-up, push-up) until rep counting and form scores are stable across a few different people/camera angles.
4. Output format: standardize every processed rep into one JSON object, e.g.:

```
{  
  "exercise": "squat",  
  "rep_number": 4,
```

```

    "timestamp_start": 1234.56,
    "timestamp_end": 1235.90,
    "form_score": 82,
    "depth_pct": 94,
    "symmetry_offset": 3.1,
    "flags": ["slight forward lean"]
  }

```

This standard shape is what the fusion layer will consume — decide it now so every exercise module outputs the same structure.

Step 2 — Build the IMU firmware (parallel track, hardware team)

1. Clone **Wojtek120/IMU-velocity-and-displacement-measurements**, flash it to the **Seeed XIAO nRF52840 Sense**.
2. Adapt its velocity/displacement math (ZUPT-based drift correction) to your board's exact IMU (LSM6DS3TR-C) if the pin/library mapping differs.
3. Add a custom BLE GATT characteristic that streams a rep-summary object (not raw 100Hz data — too much BLE bandwidth) once per rep:

```

    {"rep": 3, "peak_velocity": 0.82, "avg_velocity": 0.61, "rom_mm":
410}

```

4. Validate this against **dw2kim/BarbellTrackingCode** run on the same set (film the same reps, compare optical vs IMU velocity) before trusting your own numbers — every reference project above did this exact validation step before shipping.

Step 3 — Build the mobile app shell

1. Scaffold a React Native app.
2. Wire up **react-native-ble-plx**: scan → connect → subscribe to your custom characteristic from Step 2. Reference OpenBarbell-V3's BLE data-pipeline docs for characteristic/service UUID conventions.
3. Port your Step 1 pose pipeline into the app. Two practical options:
 - **On-device (recommended for demo reliability)**: use MediaPipe's native mobile SDK (iOS/Android bindings) and reimplement your config-driven exercise engine logic in the app itself — no network dependency, works offline.
 - **Backend-served (faster to build, needs network)**: keep yakupzengin's Flask backend as-is, have the app stream camera frames to it over WebSocket and receive rep JSON back — much less porting work, good enough for a hackathon demo if venue WiFi/hotspot is reliable.

Step 4 — Build the fusion layer (this is your original work — no repo covers this)

1. In the app, maintain two buffers: incoming pose-layer rep objects (Step 1 format) and incoming IMU rep objects (Step 2 format), both timestamped.
2. On each new rep event from *either* source, check if a same-exercise rep exists in the other buffer within a small time window (e.g. $\pm 1.5s$) — if the exercise is a barbell lift and both fired, merge

them into one combined rep card; if it's a bodyweight exercise, only the pose-layer object will ever exist, and that's expected — just skip the merge step in that case.

3. Combined rep card shape:

```
{
  "exercise": "squat",
  "rep_number": 4,
  "form_score": 82,
  "depth_pct": 94,
  "peak_velocity": 0.82,
  "flags": ["slight forward lean"]
}
```

4. Render this as a single card per rep in the UI, plus a running session summary (avg velocity trend, form score trend, rep count) per exercise.

Step 5 — Add the coaching layer (optional, high demo impact, low effort)

Feed the combined rep card's structured fields (not raw sensor data) to an LLM via the Anthropic API to generate one natural-language coaching line per rep or per set, e.g. turning {"form_score":82,"flags":["slight forward lean"]} into "Good depth this set — watch the forward lean creeping in on your last two reps." This is safe because the LLM is only summarizing numbers you've already computed, not diagnosing anything on its own.

Step 6 — Validate end-to-end

- 1. Run a full set of each target exercise (squat with IMU, pull-up/push-up camera-only) through the whole pipeline.
- 2. Check: does the fusion layer correctly merge barbell reps and correctly *not* try to merge bodyweight reps? Does the UI clearly show "no velocity data" for bodyweight exercises rather than a blank/broken field?
- 3. Test under demo-realistic conditions: gym lighting, phone on a wobbly tripod, a lifter who doesn't rack the bar perfectly — this is where most of your remaining debugging time goes, not in the algorithms themselves.

4. What you're actually building vs. what's reused

Component	Status
Pose estimation engine	Reused (MediaPipe/YOLO SDK)
Per-exercise rep counting logic	Reused + lightly extended (from Layer A repos)
IMU velocity math	Reused + adapted to your exact board (from Layer B repos)
BLE plumbing	Reused (Layer D libraries)
Fusion layer (camera + IMU → one rep card)	Original — no repo covers this
Config-driven multi-exercise extension	Mostly original, following yakupzengin's pattern
Coaching text layer	Original (thin LLM wrapper over your own structured

App UI/UX, session history, onboarding/calibration	data) Original
---	---------------------------------

The reused pieces get you a working demo fast; the bolded original pieces are where your actual product value — and your defensible pitch — lives.